

Maestría en Economía
Facultad de Ciencias Económicas -- Universidad Nacional de La Plata

Seminario de Stata

clase 4

Santiago Garganta
(CEDLAS-UNLP)

Universidad Nacional de La Plata
2017

Clase anterior

- Análisis de gestión de base de datos
 - merge
 - append
 - reshape
 - Collapse
- Datos experimentales

Clase 4

- Datos experimentales
- Regresiones
- Comandos post-regresión
- Macro locales
- Matrices
- Bucles
- Argumentos (organización modular)
- Ejercicios

Datos experimentales

- Stata permite crear datos experimentales
- En este caso los vamos a utilizar para luego aprender el manejo de algunos comandos y luego regresiones
- En primer lugar se debe indicar la cantidad de observaciones que va a contener la base de datos
- Luego, se generan las variables

Datos experimentales (clase anterior)

- Construir una base de datos con 150 observaciones
 - clear
 - set obs 150
- Crear una variable de tipo fecha que contenga la identificación de días desde el 18/05/1983 en adelante
 - gen fecha = date("18/05/1983","DMY")
 - ???

Datos experimentales (Ej. 2)

- Construir una base de datos con 1000 observaciones
- Generar dos variables aleatorias distribuidas de manera uniforme (x , w)
- Generar otra variable aleatoria (u) cuyos valores surgen de una distribución Normal (0,1)
- Fijar la semilla antes de crear variables aleatorias para luego poder replicar los resultados

Datos experimentales

- Generar $y = 4 + 2.6 * x + 1.1 * w + u$
- Borrar u
- Regresión y x (y x w)
- Comando regress indica que la primera variable representa el término dependiente del modelo y las siguientes son los términos independientes

Datos experimentales

- Comandos Post regresión
 - **regress y x w**
 - **predict yhat** (permite obtener el valor estimado de y de acuerdo al modelo de regresión)
 - Se puede obtener lo mismo de la siguiente manera:
gen yhat1=_b[_cons] + _b[x]*x + _b[w]*w
 - Los operadores **_b[]** permiten acceder a los valores estimados de los parámetros de la regresión

Datos experimentales

- Comandos Post regresión
 - El comando **predict** también nos permite obtener los errores de la regresión
 - **predict** errores, resid
 - Se puede obtener lo mismo de la siguiente manera:
gen errores1=y – yhat
 - Comparar

Datos experimentales

- Comandos Post regresión
 - El comando **test** permite realizar pruebas de hipótesis lineales sobre los parámetros estimados de la regresión
 - Ejemplo: test x=0
 - (1) x = 0
 - F(1, 997)=548.31
 - Prob > F=0.0000
 - Esto indica que se rechaza la Hipótesis nula de que el coeficiente de x=0

Datos experimentales

- Comandos Post regresión
 - Esto indica que se rechaza la Hipótesis nula $x=0$
 - Este resultado ya se encontraba en la salida de regresión. Observar que el estadístico t de x en la salida de regresión es 23.42 (valor correspondiente a una distribución t). Si la distribución F es una t al cuadrado, podemos comprobar que 23.42 al cuadrado es aproximadamente 548
 - Otros test: test $x=2.5$, test $x+w=3.7$

Datos experimentales

- Grafico:
 - regress y x
 - Obtener el y estimado
 - Graficar

- regress y x w
- Obtener el y estimado
- Graficar

Macros Locales: Una Introducción

- Ejercicio:
 - generar dos variables aleatorias x y y distribuidas uniformemente entre 0 y 1
 - generar una tercera variable como
$$z = (x + y) / (\mu_x + \mu_y)$$

Ejercicio

- Se pide (utilizando EMNV 98 de Nicaragua):
 - calcular la tasa de pobreza empleando una LP de 324.964 (=2.5 US\$ diarios a PPA)
 - Utilizar una macro local para la línea de pobreza

Ejercicio

- Se pide (utilizando EMNV 98 de Nicaragua):
 - calcular la tasa de pobreza empleando una LP de 324.964 (=2.5 US\$ diarios a PPA)
 - Utilizar una macro local para la línea de pobreza
 - Hacer el mismo cálculo para la zona rural y urbana
 - Mostrar los resultados con una tabla

Ejercicio

- Resultado
 - Pobreza Total: 48.87%
 - Pobreza Urbana: 37.86%
 - Pobreza rural: 61.88%

Include vs do

- El comando `do` ejecuta un archivo `.do`
- El comando `include` representa un copy-paste de lo que contiene un archivo `.do`. Es decir, el contenido del último al archivo `.do` pertenece al `.do` donde se ejecuta el comando *include*
- Ver diferencias (relevancia de las mismas cuando se trabaja de forma modular)

Matrices

- Muy útiles para acumulación y visualización de resultados
- Crear matriz vacía
- Crear matriz con su contenido
- Asignar valores a celdas particulares de la matriz
- Operaciones con matrices
- Vincular matrices
- Eliminar matrices

Ejercicio de Pobreza (Nicaragua)

- Cálculo de la tasa de pobreza en Nicaragua
- Tasa de Pobreza Total, Urbana y Rural
- Acumular los resultados en una matriz
- Mostrar los resultados contenidos en la matriz
- Se puede exportar la matriz en otro formato (.xls)
- Antes se crean variables a partir de la matriz (svmat) y luego se exporta.
- Ver mkmat

Bucles (Loop)

- Nos permiten realizar un mismo calculo varias veces (iterando sobre diferentes valores o elementos)
- Entre {} expreso lo que se repite. Qué y cuantas veces lo voy a repetir lo determina el tipo de bucle
- Diferentes tipos de bucles:
 - foreach
 - forvalues
 - While
- Ver Ejemplos

Ejercicio a

- El siguiente ejercicio combina los conceptos analizados anteriormente:
 - Utilizar la base nicaragua 1998
 - Calcular el ingreso laboral promedio en cada region (4 en total)
 - Acumular los resultados en una matriz
 - Mostrar resultados

Ejercicio b

- El siguiente ejercicio combina los dos conceptos analizados anteriormente:
 - Generar una matriz de 2x3 (elegir los valores)
 - Mostrar el valor de la celda [2,3]
 - Mostrar todos los valores iterando por fila y columna